



C PROGRAMMING LANGUAGE OPERATORS

PART 2

TYPES OF OPERATORS

Relational operators

Logical operators

Bit wise operators

Conditional operators (ternary operators)

Increment/decrement operators

Special operators



Relational Operators

used to find the relation between two variables. i.e. to compare the values of two variables in a C program.

also known as comparison operators
answerable by **True or False** or **Yes or No**

Symbols	Name
<	less than
>	greater than
<=	less than or equal to
>=	greater than or equal to
!=	Not equal to
==	Equal to

Analyze the given table using relational operators

Example: A = 5 B = 3		
Operator	Expression	Answer
<	A < B	
>	A > B	
<=	A <= B	
>=	A >= B	
!=	A != B	
==	A == B	



Logical Operators

these operators are used to perform logical operations on the given expressions.

answerable by **True or False** or **Yes or No**

Symbols	Name
&&	AND
	OR
!	NOT

Analyze the given table using logical operators

TRUTH TABLE			
Expression	AND output (&&)	OR output ()	NOT (!)
True && True	True	True	!True = False
True && False	False	True	
False && True	False	True	!False = True
False && False	False	False	

Example: A = 5 B = 2	
Expression	Answer
(A < B) && (B!=A)	
(A == B) (A >= B)	

Analyze the given expression using different types of operators

Example: A = 5 B = 2 C = 10

Expression	Answer
$(A * B) < C \parallel (C \% A == B)$	
$!(((A + C) - B) >= ((C * B) / A)) \&\& (C / B * A <= B * A)$	



Bitwise Operators

Mathematical operations like addition is converted into bit-level which makes processing faster and saves power.

Symbols	Name
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
~	Bitwise complement
<<	Shift left
>>	Shift right

Bitwise Operators

```
    1010
&   0110
-----
    0010
```

AND Operator (&): The AND operator compares two bits and returns 1 if both bits are 1, otherwise returns 0.

Bitwise Operators

OR Operator (|): The OR operator compares two bits and returns 1 if at least one of the bits is 1, otherwise returns 0

```
      1010
    | 0110
    -----
      1110
```

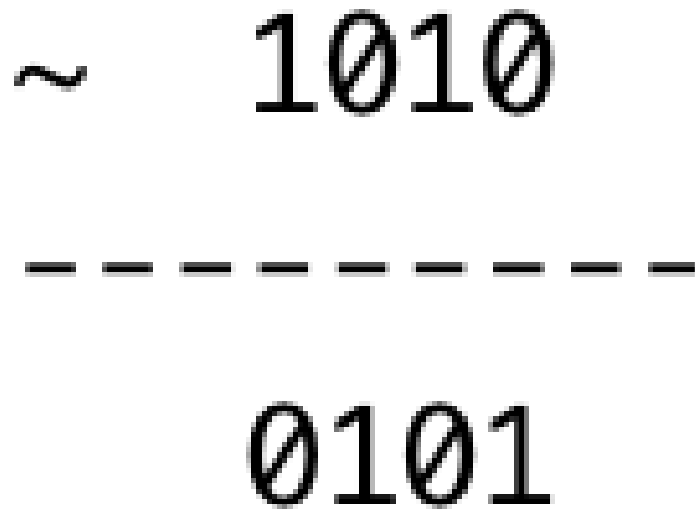
Bitwise Operators

```
      1010
^     0110
-----
      1100
```

XOR Operator (^): The XOR operator compares two bits and returns 1 if the bits are different (one 0 and the other 1), otherwise returns 0.

Bitwise Operators

Complement Operator (~): The complement operator flips the bits, turning 0 to 1 and 1 to 0.



The diagram illustrates the complement operation. It features a white rectangular box with a thin blue border. Inside the box, the text is arranged as follows: the tilde symbol (~) is positioned to the left of the binary number 1010. Below 1010 is a horizontal dashed line consisting of eight dashes. Below the dashed line is the resulting binary number 0101. This visualizes the process of flipping each bit of the original number.

~ 1010

0101

Bitwise Operators

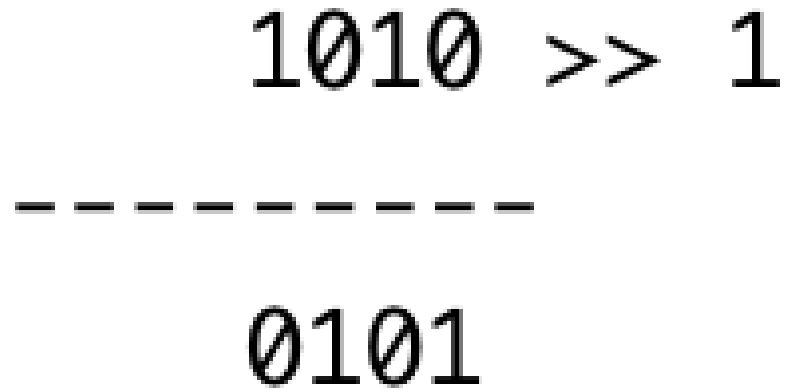
1010 << 2

101000

Left Shift Operator (<<): The left shift operator shifts the bits of a number to the left by a specified number of positions, effectively multiplying the number by a power of 2

Bitwise Operators

Right Shift Operator (>>): The right shift operator shifts the bits of a number to the right by a specified number of positions, effectively dividing the number by a power of 2



A diagram illustrating a right shift operation. It shows the binary number 1010 shifted one position to the right, resulting in 0101. A vertical line is on the left, and a dashed line separates the original number from the result.

$$\begin{array}{r} 1010 \\ \text{-----} \\ 0101 \end{array} \gg 1$$



Increment / Decrement Operators

Mathematical operations like addition is converted into bit-level which makes processing faster and saves power.

Symbols	Name
++	Increment
--	Decrement



Conditional Operator

This is also known as ternary operator that can work on 3 operands.

- Syntax: conditionalExpression ? expression1 : expression2
- The first expression conditionalExpression is evaluated first.
- This expression evaluates to 1 if it's true and evaluates to 0 if it's false.
 - If conditionalExpression is true, expression1 is evaluated.
 - If conditionalExpression is false, expression2 is evaluated.

Symbols	Name
?	Increment

Sample Program for Conditional Operator

```
1. #include <stdio.h>
2. int main()
3. {
4.   char February;
5.   int days;
6.   printf("If this year is leap year, enter 1. If not enter any integer: ");
7.   scanf("%c",&February);

8.   // If test condition (February == '1') is true, days equal to 29.
9.   // If test condition (February == '1') is false, days equal to 28.
10.  days = (February == '1') ? 29 : 28;

11.  printf("Number of days in February = %d",days);
12. }
```

◦

